RFI Respond: drafting from what the organisation already knows

An internal tool that answers procurement questionnaires from an organisation's own knowledge base. It drafts each answer from cited sources, says how confident it is and why, keeps pricing and legal questions for a person, and learns from every response it helps send.

Rory McClelland · mcclelland.info

Sprint 24 · November 2026 · code at github.com/rorymcclelland/RFI-responder

25
sprints to rebuild a prototype into a governed AI tool, each one ending in working software

305 of 305
claims in the drafts judged supported by the knowledge-base entries they cite

0 of 33
near misses (on topic, but not answered by the knowledge base) reached the bulk-approval line

1,648
automated tests: 1,173 unit, 438 database tests of access rules, approvals, and learning, and 37 browser tests of the agreed cases

Organisations that sell through formal procurement answer requests for information (RFIs), requests for quotation (RFQs), and security questionnaires again and again. Most of the questions have been answered before, but the answers sit in old documents and in people's heads. Each response is slow to assemble, its quality depends on who happens to work on it, and what one team learns is rarely reused by the next.

That makes the RFI response a good example of a whole class of business process: knowledge-heavy, repetitive, high-stakes in places, and owned by several people at once. I built RFI Respond to show how AI can be designed into a process like this to make it **faster**, **more accurate**, and **more repeatable**, without handing any decision that matters to a model. I owned the product and made every decision; Claude, as an AI coding agent, wrote the code, and I read and questioned every change. It runs as an invite-only demo on synthetic data only, where a visitor can try the whole job on a sample questionnaire of their own, and the full code and documentation are public.

01

From a three-hour prototype to a working tool

Built for a job interview, in a three-hour time box.

RFI Respond began as a prototype built for a job interview, with three hours on the clock. The aim in that time was to make the whole workflow visible and usable end to end, so the intelligence was simulated: questions were matched to past answers by keywords, drafts were assembled from templates, and the confidence score was a placeholder. It did its job, giving the conversation a working product to point at.

This build replaces the simulation with the real thing. The workflow stayed, and every part that stood in for intelligence was rebuilt as a part that can be measured. That is the same step most early AI demonstrations have to take before an organisation can rely on them.

Step	Done by hand	With RFI Respond
Read the questions	Copied out of each customer's spreadsheet or Word document	Read from either format; in Word, a model finds the questions and code keeps only those found word for word, for a person to check
Find past answers	Search old responses, or ask around	Matched by meaning across the knowledge base, in English or Japanese
Draft	Rewrite an old answer to fit	Drafted only from the matched entries, citing them; no draft when nothing answers the question
Decide what can go	Every answer checked one by one, or not at all	Calibrated confidence and a stated reason; the owner approves in bulk above a threshold, within limits the database enforces
Specialist questions	Chase whoever might know	Routed to colleagues suggested from their expertise and what they have answered before
Learn	The final answers stay in the sent file	Edited and person-written answers become knowledge-base entries, with their source, when the response is marked as sent

02

Faster, more accurate, more repeatable

Each of the three aims is built into a different part of the design. Accuracy and repeatability are measured; speed is designed for, and would be measured against a baseline in a real deployment (section 07).

Faster

- Every question gets a first draft, or a clear reason why not, before a person opens it
- Review starts with the lowest confidence, where a person's time matters most
- Answers the evidence supports are approved in one recorded action
- Specialist questions go straight to a named colleague's inbox

More accurate

- Drafts come only from cited entries, and code rejects any figure, link, or email address not in them
- A separate check, before drafting, confirms that the entries actually answer the question
- Confidence is measured against an evaluation set, not guessed
- Pricing and legal questions always go to a person

More repeatable

- The same pipeline, models, and prompt versions every time, recorded with every answer
- Every export says how each answer was produced and who approved it
- The knowledge base improves with each response, with provenance and review dates
- Every evaluation can be re-run with one command

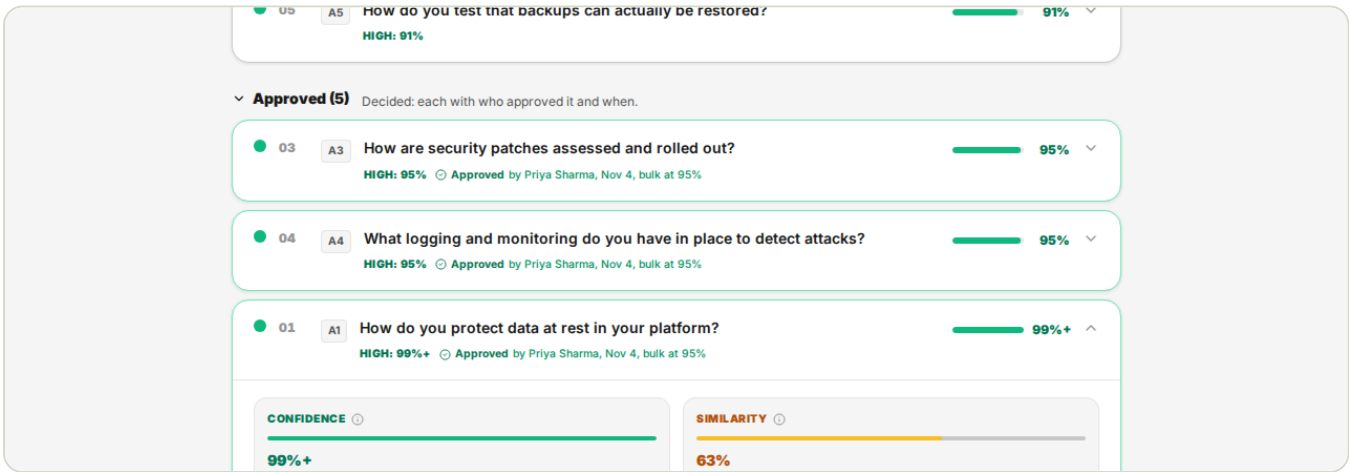
03

How the AI works

Each question passes through the same server-side pipeline. Models are used where judgement about language is needed: matching, checking, classifying, and drafting. Everything that must be exact is ordinary code, which behaves the same way every time and can be tested exhaustively. Lime steps use a model; dashed steps are code; the dark step is a person.



One question, from upload to export. Models are kept to the steps that need them, and every one of their outputs is checked by code or a person before it can leave.



A response in review, with synthetic data: drafts nothing holds wait for bulk approval, most confident first; answers approved in bulk at 95% carry who approved them and when; a Japanese question is labelled as a translated draft; and an open answer shows its calibrated confidence beside raw similarity.

04

Six AI design choices

Each choice is written up as a decision record, with the evidence that settled it.

The interesting decisions in an AI product are rarely about which model to use. They are about what the model is allowed to decide, how its output is checked, and what happens when it is unsure.

Models · decision 0005

Chosen by measurement, not by leaderboard

Three embedding models were compared on the same English and Japanese questions before one was picked. Matching sits behind a provider switch: a hosted embedding model by default, or a local one on the local PC, so questions and entries can be matched without leaving it. Checking and drafting use Claude in both set-ups.

Result voyage-4 found the right entry in the top five for 99.5% of English questions and 100% of Japanese ones asked against English entries.

Answerability · decision 0007

Similar is not the same as answered

Similarity alone cannot tell an entry that answers a question from one on the same topic. A checker model reads the entries and says whether they answer it, with a reason a person can read.

Result With the checker, near misses are separated from answerable questions almost perfectly (AUROC 0.997, against 0.892 for similarity alone).

Confidence

A number that means what it says

Confidence is calibrated on the evaluation set: when the tool says 80%, the knowledge base answers the question about 80% of the time. It is shown as "99%+" at the top, never as certainty.

Result Calibration error 0.047, checked by five-fold cross-validation; no near miss reached the 80% bulk line.

Grounding

Model output is never trusted on its own

The drafter writes only from the entries it was given. Code then rejects a draft that cites anything else or introduces a figure, link, or email address its sources do not contain. Unreadable model replies count as failed checks, never as a pass.

Result All 305 claims in the test drafts judged supported by their sources; the judge itself caught 95 of 95 unsupported claims planted to test it.

Gates

Some questions are never the model's

Pricing and legal or contractual questions are caught by rules and a classifier together, and are always decided by a person. Pricing is never drafted and never learned, because prices go stale.

Result 100 of 100 pricing and legal questions gated; 7 of 70 lookalikes gated by mistake, a cost accepted because the error falls on the side of a person.

Learning

The knowledge base learns, with provenance

When a response is marked as sent, edited and person-written English answers become entries linked to the response they came from. An answer may replace a near-duplicate only above a measured similarity, and a member's replacement waits for an admin's review.

Result At the chosen threshold, wrong replacements came to 1.9% of test questions (7 of 363), against a bar of 2%; the price is that only about half of true duplicates are caught.

05

Evidence, including what didn't pass

In the repository, a test checks that every figure in the evaluation summary matches its report.

Every claim about the AI is backed by an evaluation that can be re-run. The main evaluation set has 363 questions written from the synthetic knowledge base, including paraphrases, Japanese translations, unanswerable questions, and near misses; drafting, the gates, and the red team use smaller sets of their own. Faithfulness is judged by a second, larger model, spot-checked against a person's review. Each figure carries a 95% interval, because a small set gives a range, not a point.

Measure	Figure	95% interval
Right entry in the top five, English (recall@5)	**99.5%**	97.2 to 99.9%
Right entry in the top five, Japanese to English	**100.0%**	96.3 to 100.0%
Calibration error (cross-validated)	**0.047**	0.025 to 0.075
Near misses at or above the 80% bulk line	**0 of 33**	0.0 to 10.4%
Pricing and legal questions gated	**100 of 100**	96.3 to 100.0%
Claims in drafts judged supported by the cited entries	**305 of 305**	92.3 to 100.0%
Right entry cited	**93.9%**	83.5 to 97.9%
Checker verdict changed when asked again (five runs)	**7.2%**	4.2 to 11.9%

What the evaluations changed

The value of measuring is that it changes the build. Blending keyword and meaning-based search lowered English retrieval by about 20 points for every model tested (voyage-4: 99.5% to 80.0%), so matching went meaning-first. When the checker gave different verdicts on repeated runs, the investigation in Sprint 6 found that about a third of its replies were being cut off and counted as "no"; after the fix, every figure that depends on the checker was measured again. It still changes its verdict for 7.2% of questions when asked again, against a bar of 5%. That miss was accepted as a residual, since most changes are between "yes" and "partly" and only 1.7% crossed the bulk line. Separately, "partly" verdicts cited the wrong entry twice as often as "yes" (15% against 7%), so in Sprint 7 bulk approval was changed to need a firm "yes".

Attacking it

Questionnaires and knowledge-base entries are text written by other people, so they are treated as data, never as instructions. A red team tried 37 prompt injections. As first scored, two counted as successes; one was the drafter refusing and naming the standard it refused to claim, and the scoring rule was narrowed after the run, recorded as such. The other was real: an instruction moved the checker's verdict on a near miss from "no" to "partly". The drafter refused to answer it, so there was nothing to approve, and since Sprint 7 "partly" is never bulk-approved. Four Word documents carrying planted instructions changed nothing.

06

How it was built

The rebuild has run as twenty-five sprints, from the original 65 job stories and a one-page product brief that settles disputes: where a story and the brief disagree, the brief wins until it is changed. Each sprint follows the same cycle: refine the stories into test cases, risks, and questions for the product owner; build test-first; from Sprint 2 onwards, have a separate AI agent, which had not seen the reasoning, review the changes for security and correctness; check end to end; and close, returning anything undelivered to the backlog. Review findings are fixed with a test in the sprint, or recorded as a residual with the product owner's agreement and added to the backlog.

Since Sprint 20, Claude has also been the QA team. Every agreed test case becomes a browser test that uses the product as a person would, run on every push against a local copy and, on each deploy, against a hosted staging copy where the real models draft a sample questionnaire. A second agent checks that the tests really prove the cases. I sign off each sprint from a demo page of the recordings, as I would at a QA team's sprint review, and the live demo is deployed only after my approval.

Sprint	What it delivered
0	Clean-up: access rules in the database, tests, CI, and honest copy in place of invented figures
1 · 2	Matching by meaning and calibrated confidence; grounded drafting, the answerability check, pricing and legal gates, and server-side processing
3 · 4	Routing and the learning loop; spending caps, entry review, and the governance pack
5 · 6	Word questionnaires; the checker's reliability
7 · 8 · 9	Documentation, demo data, and publishing; 95% intervals on every figure; bulk approval tightened in every set-up, and the faithfulness judge tested on planted claims
10 to 15	A user guide in the app; a knowledge base in English and Japanese; an invite-only hosted demo
16 to 19	A UX spike, then the app reorganised around people's jobs: Home, the knowledge base as a place of its own, an answer's own page, asking the knowledge base a question, and a colleague's questions answered on one screen
20 to 24	Claude as the QA team; gaps in the knowledge base; the response as a journey in three stages; Try it with a sample questionnaire; one list across a response's files

Governance was built in rather than written up afterwards: a model card, a risk register that names the tests behind each risk and marks the controls not yet built, a data-handling note, and a mapping of the controls to the EU AI Act and the NIST AI Risk Management Framework, with no claim of compliance. Spend is capped too: every model call is recorded and priced, with a daily limit per person and a monthly budget.

07

The same pattern, in other processes

The RFI is the example. The pattern is the point.

Take away the procurement detail and what remains is a reusable shape for any process where people answer the same kinds of question from the same body of knowledge:

1. **Capture** the institutional knowledge, with provenance and review dates.
2. **Match** each new request to it by meaning, in any language the organisation works in.
3. **Check** whether the knowledge actually answers the request, and say why.
4. **Draft** only from that knowledge, and verify the draft in code.
5. **Gate** the categories a model must never decide, and calibrate the rest.
6. **Let people decide**, in bulk where the evidence allows and one by one where it does not.
7. **Learn** from what people finally sent.

Process	Knowledge base	Always a person
Security and due-diligence questionnaires	Policies, certifications, past questionnaires	New commitments, exceptions
Customer support replies	Help articles, resolved tickets	Refunds, complaints, account changes
HR and policy questions	Handbook, policies, past answers	Anything about an individual
Supplier onboarding	Company facts, standard terms, past forms	Bank details, contract terms
Bids and grant applications	Past submissions, case studies	Budgets, pricing, eligibility claims
Sustainability (ESG) reporting	Published reports, measured data	Any new figure or target

In each case the questions to settle first are the ones settled here: which part of the work is assembling context, which part is judgement, which categories are never the model's, and what counts as good enough to approve in bulk. They map directly onto the method I use to choose and prove AI work in an organisation: find where time goes on gathering context and waiting for one person's judgement, record a baseline before anything changes, prove the workflow on a contained slice, and reuse the foundation for the next one.

What a real deployment would add

This is a portfolio build on synthetic data, so its figures are estimates on small sets. A real deployment would start with a baseline (the time from receiving a questionnaire to sending a complete response, and the share of answers changed in review), then an evaluation set drawn from the organisation's own past questionnaires, and the hosting and access steps already written down in the repository. The time saved would then be measured against that baseline, rather than claimed.

08

What it shows

Most of the effort in an AI product goes into the parts that are not AI: deciding what the model may not do, checking what it produces, measuring whether it is right, and making every decision traceable to a person. That is what turns a convincing prototype into something an organisation could trust with a customer-facing answer, and it is the part that transfers from one process to the next.

Built with: React and TypeScript, Supabase (Postgres with row-level security, Edge Functions, and pgvector), Claude Haiku 5.5 for checking, classifying, drafting, and finding questions, voyage-4 or a local embedding model through Ollama, Vercel for the demo, and Playwright for the browser tests. All rights reserved; the code is public to read at github.com/rorymcclelland/RFI-responder.

Rory McClelland · product leader, AI products and governance
hello@mcclelland.info · mcclelland.info

Companion case studies:
How I build with AI · **Job Scout** · **Task oversight**